

# Experiment 10

Student Name: Bhavik Pahuja  
Branch: CSE - AIML  
Semester: 4  
Subject Name: DBMS

UID: 24BAI70522  
Section/Group: 24AIT\_KRG G1  
Date of Performance: 24/4/26  
Subject Code: 24CSH-298

## Aim

To design a trigger that automatically implements the functionality of a primary key, ensuring unique identification of records without manual intervention.

## Software Requirements

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

## Objectives

- To create a database trigger that automatically enforces primary key constraints or generates unique key values, replicating the functionality of a stored procedure.

## Problem Statement

- In enterprise databases, primary keys must be unique for every record. Manual assignment of keys can lead to errors.
- Design a trigger that:
  - Automatically generates or enforces a primary key value during record insertion.
  - Implements the logic similar to a stored procedure.
  - Demonstrates automated primary key functionality for a table.

## Practical/Experiment Steps

- Schema Provisioning: Created the employees table with a structured format for personal and departmental data, specifically focusing on the emp\_id as the unique identifier.
- Sequence Generation: Established an independent Database Sequence (emp\_seq) to provide a continuous and reliable stream of unique numerical values.
- Trigger Function Development: Authored a PL/pgSQL function designed to intercept incoming data and check for the presence of a primary key value.
- Automated Logic Injection: Configured the function to automatically fetch the next value from the sequence using nextval() whenever a record is inserted without an ID.
- Before-Insert Activation: Bound the function to the table using a BEFORE INSERT trigger, ensuring that identity assignment occurs prior to data storage to prevent null constraint violations.
- Validation Testing: Conducted multiple insertions without specifying IDs to verify the trigger's ability to maintain data integrity automatically.

## Procedure

- Logged into the PostgreSQL environment and initialized the database session for the experiment.
- Executed the CREATE TABLE command to define the employee structure.
- Initialized a Sequence starting at 1 to serve as the source for automated ID generation.
- Defined the auto\_emp\_id() trigger function to act as the primary key generator.
- Implemented conditional logic within the function to ensure that if a user does not provide an emp\_id, the system assigns one dynamically.
- Created the trg\_auto\_emp\_id trigger and attached it to the employees table for each row.
- Performed test insertions by providing only the name and department, leaving the primary key field blank.
- Executed a SELECT \* query to verify that the system successfully assigned sequential IDs (1, 2, etc.) to the new records.
- Documented the results to demonstrate how the trigger replicates the functionality of auto-incrementing fields used in companies like Amazon and Oracle.

# Input/Output Analysis

## SQL Input Queries

```
CREATE TABLE employees (  
    emp_id INTEGER PRIMARY KEY,  
    emp_name VARCHAR(100),  
    department VARCHAR(50)  
);
```

## Output

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 123 msec.

## SQL Input Queries

```
CREATE SEQUENCE emp_seq  
START 1;
```

## Output

Data Output Messages Notifications

CREATE SEQUENCE

Query returned successfully in 135 msec.

## SQL Input Queries

```
CREATE OR REPLACE FUNCTION auto_emp_id()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.emp_id IS NULL THEN  
        NEW.emp_id := nextval('emp_seq');  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

## Output

Data Output [Messages](#) Notifications

```
CREATE FUNCTION
```

Query returned successfully in 100 msec.

## SQL Input Queries

```
CREATE TRIGGER trg_auto_emp_id  
BEFORE INSERT ON employees  
FOR EACH ROW  
EXECUTE FUNCTION auto_emp_id();
```

## Output

Data Output [Messages](#) Notifications

```
CREATE TRIGGER
```

Query returned successfully in 106 msec.

## SQL Input Queries

```
INSERT INTO employees (emp_name, department)  
VALUES ('Alice', 'HR');
```

```
INSERT INTO employees (emp_name, department)  
VALUES ('Bob', 'IT');
```

## Output

Data Output [Messages](#) Notifications

```
INSERT 0 1
```

Query returned successfully in 96 msec.

## SQL Input Queries

```
SELECT * FROM employees;
```

## Output

| Data Output   |                        |                                     |                                      |
|---------------|------------------------|-------------------------------------|--------------------------------------|
| Messages      |                        |                                     |                                      |
| Notifications |                        |                                     |                                      |
| SQL           |                        |                                     |                                      |
|               | emp_id<br>[PK] integer | emp_name<br>character varying (100) | department<br>character varying (50) |
| 1             | 1                      | Alice                               | HR                                   |
| 2             | 2                      | Bob                                 | IT                                   |

## Learning Outcomes

- Trigger-Based Automation: Proficiency in designing triggers that eliminate the need for manual primary key assignment during data entry.
- Sequence Management: Understanding how to create and utilize database sequences for generating guaranteed unique identifiers.
- Data Integrity Enforcement: Mastery of 'Before-Action' triggers to intercept and validate records before they are committed to the database.
- Backend Logic Encapsulation: Gained the ability to implement enterprise-level automation where the database handles identity management independently of the application layer.